Needl setup manual

Connect seven tools twice: to Claude Code (MCP) and to Needl (webhooks)

GitLab · Linear · LaunchDarkly · Sentry · PostHog · Intercom · Stripe

Prepared for Kavish · written 2026-10-19 against Needl commit 16eec52 and vendor docs checked that day

How to read this manual

Follow it top to bottom. Each step starts with a blue label telling you which window to use:

Terminal (Mac)

Browser tab: Needl

Browser tab: *provider*

Bold text is the exact label on screen. `Monospace` is something you type or paste. Grey boxes are commands or prompts to copy. Yellow **Copy & paste** boxes say what to move between windows. Red boxes are stop points: do what they say and don't improvise.

Copy & paste

Values that never change:
Needl address: `https://needl.dev`
Needl Connections page: `https://needl.dev/#/connections`
Every webhook URL comes from that page's **Copy** button. It looks like `https://needl.dev/api/hooks/int_stripe_a1b2c3d4e5f6`. The last part is unique to your workspace, so never type it by hand.

Contents

1 Before you start . . . 2
2 One-time preparation . . . 4
3 GitLab . . . 7
4 Linear . . . 9
5 LaunchDarkly . . . 11
6 Sentry . . . 13
7 PostHog . . . 15
8 Intercom . . . 18
9 Stripe (sandbox only) . . . 21
10 After setup . . . 24

1 Before you start

1.1 Two separate connections per tool

Each tool gets two connections, and they don't know about each other:

	A. MCP (tool → Claude Code)	B. Webhook (tool → Needl)
What it does	Lets Claude Code on your Mac read (and, if you allow it, change) the tool for you.	The tool sends events to Needl, Needl checks the signature, and Needl runs your rules.
Where you set it up	Mac Terminal plus a browser sign-in page	The tool's settings page plus Needl's Connections page
Credential	OAuth sign-in, stored by Claude Code on your Mac	A signing secret, saved encrypted in Needl
Rights you need	Your normal user account	Usually an admin or maintainer role in the tool (see 1.3)

Finishing Part A does **not** set up Part B, and the reverse is also true. In this manual every webhook is created by hand in the tool's own interface, even when its MCP could do it.

1.2 Order of work

1. Read the checklist in 1.3 and 1.4. Where the last column applies to you, do that first or skip that tool for now.
2. Do Section 2 once to get Terminal, Claude Code and Needl ready.
3. Work through Chapters 3–9 one at a time, in any order. Inside a chapter, do A, then B, then C. Section D is only for when something goes wrong.
4. Tick the chapter's success line before you start the next one.

1.3 Accounts and permissions checklist

Tool	A. MCP needs	B. Webhook needs	If you lack it
GitLab	GitLab.com account. Your top-level group must allow MCP (**Allow connection to GitLab**, which is on by default).	Webhook already exists (Chapter 3). Creating a new one needs **Maintainer** or **Owner** on the project.	You are **Developer** on the hackathon project. Ask a Maintainer/Owner (message in 3.D).
Linear	Any member of the workspace.	**Workspace admin**.	Ask a workspace admin to do 4.B.
Launch-Darkly	Role that can manage flags (LaunchDarkly suggests **Writer** or **Developer**). US commercial instance only: no EU or federal.	Permission to add organisation integrations.	Ask a LaunchDarkly admin or owner to do 5.B.

Tool	A. MCP needs	B. Webhook needs	If you lack it
Sentry	Member of the Sentry organisation.	Permission to create Custom Integrations in Organisation Settings.	Ask a Sentry org Owner or Manager to do 6.B.
PostHog	Member of the PostHog project.	Data pipelines → Destinations available. The HTTP Webhook destination is a paid template.	Ask the billing admin to enable Data pipelines.
Intercom	Teammate in a US- or EU-hosted workspace. Australian workspaces are not supported.	Access to the Developer Hub plus an app installed in the workspace.	Ask an Intercom admin.
Stripe	Dashboard user. Approve **sandbox only**.	Workbench access in a **sandbox**.	Ask a Stripe admin to add you to a sandbox.

1.4 Stripe: sandbox only

Do everything for Stripe in a **sandbox** (Stripe's isolated test environment). Sandbox payments are not processed by card networks, so nothing is charged. Live mode never comes up in this manual. If the Stripe Dashboard doesn't show a sandbox name in the account picker (top left), switch to one before Chapter 9.

1.5 Rules for secrets

- Paste secrets only into the two fields this manual names: the tool's field and Needl's secret field. Don't paste them into chat, Claude, notes or git.
- Needl's secret field is a password field. After **Save and start receiving**, Needl never shows the secret again. To change it, paste a new one; the button then reads **Replace secret**.

2 One-time preparation

2.1 Terminal and Claude Code

Terminal (Mac)

Step 1. Press **Cmd+Space**, type `Terminal`, press **Return**.

Step 2. Type the command below and press **Return**:

```
claude --version
```

Expected: a version number followed by `(Claude Code)`. If you see `command not found`, install Claude Code from https://code.claude.com/docs/en/setup and come back.

Step 3. See which MCP servers you already have:

```
claude mcp list
```

Expected: a list, which may be empty. If a name this manual uses (`gitlab`, `linear`, `launchdarkly`, `sentry`, `posthog`, `intercom`, `stripe`) already shows ✔ `Connected` with the URL this manual gives, skip that chapter's Part A.

2.2 How every Part A works (same 4 moves)

1. **Add**: one `claude mcp add ... --scope user ...` command. User scope makes the server available in every folder on your Mac. It writes only that one entry to your Claude Code settings and touches nothing else. The command is printed over two lines joined by a trailing `\`: copy both lines together and press **Return** once.
2. **Sign in**: `claude mcp login <name>`. Your browser opens the tool's consent page. Sign in, check the account or workspace shown, then click the approve button. Claude Code stores the token and refreshes it for you.
   If your Claude Code says unknown command 'login': run `claude`, type `/mcp`, press **Return**, select the server with the arrow keys, press **Return**, choose **Authenticate**, and finish in the browser. Type `/exit` afterwards.
3. **Check**: `claude mcp list` must show the server with ✔ `Connected`.
4. **Prove**: start a *new* `claude` session (sessions that were already open don't pick up new servers), paste the chapter's read-only prompt, and approve the single tool call Claude asks about. Type `/exit` when done.

To undo a Part A later: `claude mcp remove <name> --scope user`. This also deletes the stored sign-in.

2.3 Needl: open the Connections page

Browser tab: Needl

Step 1. Open https://needl.dev and sign in with the account you use for Needl.

Step 2. In the left sidebar click **Connections**, or open https://needl.dev/#/connections.

Step 3. **Expected:** the page title is **Connections**, with a **GitLab Duo flow** panel at

the top and one card per tool below it. Each card shows **Not connected**, **Sample data only**, or **1 webhook**.

Keep this tab open for the whole manual. The page refreshes itself every few seconds, so you never need to reload it.

2.4 How the Needl connect dialog behaves (all tools)

- Clicking **Connect** on a card creates that tool's own webhook route right away and opens a dialog titled **Connect** / *tool name*. The route belongs only to your workspace.
- The route accepts nothing until you save a secret. Until then the status reads **Not finished: paste the signing secret to start receiving.** Deliveries that arrive before that are refused.
- If you close the dialog early, the card button changes to **Finish setup**. Click it to reopen the *same* route; it does not create a new one. Each route row also has a **Setup** button that reopens it.
- The dialog contains: **Webhook URL** with a **Copy** button (which briefly shows **Copied**), numbered steps, the secret field, the save button **Save and start receiving**, and **Then send a test:**.
- Never click **Disconnect** in this manual. It deletes a route, and its URL stops accepting deliveries.

2.5 Needl status lines: what they mean

These are the exact texts shown on the route row and at the top of the dialog.

Needl shows	Meaning → your action
Not finished: paste the signing secret to start receiving.	No secret saved yet → finish the chapter's Part B.
Webhooks are arriving. Save the signing secret below to start verifying them.	The tool is already sending, but there's no secret → paste and save it.
Waiting for the first webhook…	Secret saved and nothing has arrived yet → do Part C.
Receiving events · last: *event*	**Success.** A signed event was verified and stored.
Receiving events · last one was a retry Needl already had	**Success.** The tool resent an event Needl already had (a duplicate).
Connected and verified, but the last event isn't one Needl reads (*x*).	The signature is fine; that event type just isn't used by Needl → OK for pings. Otherwise tick the events listed in the chapter.
A webhook arrived but its signature didn't match…	Wrong secret → see that chapter's D.
A webhook arrived with an old timestamp…	Clock or old retry → send a fresh test.
A webhook arrived but wasn't in the expected format…	Wrong webhook type, or wrong PostHog body → see that chapter's D.

Needl shows	Meaning → your action
A webhook arrived but couldn’t be stored; the provider will retry it.	Temporary Needl storage problem → wait and retest.

Delivery is not the same as action. **Receiving events** only means Needl verified and stored the event. Whether a rule broke, or a GitLab Duo flow started or finished, appears on the rule itself (its **Counterexamples** and **Investigation** tabs), not on the Connections page.

3 GitLab

Your GitLab situation (read first). GitLab push → Needl webhook → agent start already works. **Don't create, edit or delete that webhook, and don't click Disconnect or Add another on Needl's GitLab card.** The **GitLab Duo flow** panel is a different connection (Needl → GitLab, which starts flows). It is already tested, so leave it as it is. Enabling custom flows in the hackathon project is still waiting on a **Maintainer**: you are **Developer** there, and nothing in this manual changes that.

3.A Connect the official GitLab MCP to Claude Code

Endpoint `https://gitlab.com/api/v4/mcp` · transport HTTP · OAuth. Available on GitLab.com Free, Premium and Ultimate. GitLab Duo is not required to connect; the Duo tools only appear if Duo Agent Platform is available to you.

Terminal (Mac)

Step 1. Add:

```
claude mcp add --transport http --scope user \
  gitlab https://gitlab.com/api/v4/mcp
```

Expected: a line saying the HTTP server `gitlab` was added to user config.

Step 2. Sign in:

```
claude mcp login gitlab
```

Browser: GitLab authorization page

Step 3. Sign in to GitLab if asked. Check that the account shown is yours, then click the button that **authorizes** the application.

Terminal (Mac)

Step 4. Run `claude mcp list`. **Expected:** `gitlab ... ✔ Connected`.

Step 5. Run `claude`, paste this prompt, and approve the tool call:

> Using only the gitlab MCP server, tell me which GitLab user I am signed in as and list 5 projects I am a member of. Read only: do not create, change or delete anything.

Expected: your GitLab username and project paths. Type `/exit`.

The GitLab MCP has no tool for project webhooks, and connecting it gives you no new rights.

3.B Check the existing GitLab webhook in Needl (don't recreate it)

Browser tab: Needl (Connections)

Step 1. Look at the **GitLab** card, without clicking its button.

Step 2. Read the route rows under it and match what you see:

- A row with a green dot reading **Receiving events · last: push** (or another GitLab event), plus **secret saved in Needl** or **secret in server settings → done.** This is the working route. Go to 3.C.
- A row reading **Not finished: paste the signing secret…** with **no secret yet** → an unused extra route, probably from an accidental **Connect** click. It is harmless; leave it. Find the green row as above.
- The card says **Not connected** or **Sample data only** → you are probably signed in to a **different Needl account** from the one the working webhook belongs to. Sign out (Settings) and sign in with the other account, then look again. Only if no Needl account has a green GitLab row, use 3.D "New GitLab webhook".

3.C Test

No new test is needed: the green row's **last event … UTC** time already proves delivery. **Don't push commits just to test.** In this project a push can lead to a broken rule, which starts a Duo flow and uses Duo credits. If a real push happens anyway, the row should update to **Receiving events · last: push** within seconds.

3.D Fixes and handoffs

GitLab row shows a signature or old-timestamp error. The working route's secret was changed. Ask the project Maintainer to open **Settings → Webhooks**, click **Edit** on the Needl webhook, and check **Recent events**. Don't replace the secret yourself unless the Maintainer has generated a new signing token for you.

GitLab MCP says access is not allowed. The top-level group has **Allow connection to GitLab** cleared. Only a group Owner can change it: in the group, go to **Settings → General → Permissions and group features → MCP client access**, tick **Allow connection to GitLab**, then click **Save changes**.

GitLab MCP rejects your token. The MCP doesn't accept legacy personal, project or group access tokens. Use the OAuth sign-in from 3.A (`claude mcp logout gitlab`, then `claude mcp login gitlab`).

New GitLab webhook (only if 3.B found none). You need **Maintainer** or **Owner** on the project. As Developer you can't see **Settings → Webhooks**, so send the message below to a Maintainer and do the Needl side together on a call:

> Please add a project webhook for Needl: Settings > Webhooks > Add new webhook. URL: (I will send the Needl URL). Trigger: Push events, Pipeline events, Deployment events, Merge request events, Issues events. Click "Generate signing token" and paste the token into Needl's field while I share my screen, before clicking "Add webhook".

Needl side: **GitLab** card → **Connect** → **Copy** next to **Webhook URL** → select the radio option **Signing token** → paste the GitLab token (it starts with `whsec_`) into **Signing token or secret token** → **Save and start receiving** → the Maintainer clicks **Add webhook**. The Maintainer then tests with **Test → Push events**. If the error says `GitLab signing tokens start with whsec_`, the token was copied incompletely; GitLab shows it only once, so the Maintainer must generate a new one.

4 Linear

4.A Connect the official Linear MCP to Claude Code

Endpoint `https://mcp.linear.app/mcp` · Streamable HTTP · OAuth.

Terminal (Mac)

Step 1. Add:

```
claude mcp add --transport http --scope user \
  linear https://mcp.linear.app/mcp
```

Step 2. Sign in:

```
claude mcp login linear
```

Browser: Linear consent page

Step 3. Check the workspace name shown, then approve access.

Terminal (Mac)

Step 4. Run `claude mcp list`. **Expected:** `linear` … ✔ `Connected`.

Step 5. Run `claude`, then:

> Using only the linear MCP server, list the teams in my Linear workspace with their keys. Read only: do not create, change or delete anything.

Expected: your team names and keys (for example `ENG`). Type `/exit`.

Linear's MCP has no webhook tools. Webhooks need a workspace admin (4.B).

4.B Connect Linear webhooks to Needl

Browser tab: Needl (Connections)

Step 1. On the **Linear** card click **Connect** (or **Finish setup**). The dialog **Connect / Linear** opens.

Step 2. Click **Copy** next to **Webhook URL**.

Browser tab: Linear

Step 3. Open `https://linear.app/settings/api` (Settings → **API**). You must be a workspace admin.

Step 4. Find the **Webhooks** section and click **New webhook**.

Step 5. **Label**: type `Needl`. **URL**: paste (Cmd+V).

Step 6. Team: choose **All public teams**.

Step 7. Data change events: tick exactly **Issues**, **Comments** and **Issue SLAs**.

Step 8. Click the create button. The webhook's detail page opens.

Step 9. On the detail page, copy the **Signing secret**.

Copy & paste

Linear webhook detail page → **Signing secret** → Needl dialog field **Signing secret**

Browser tab: Needl (dialog still open)

Step 10. Paste into **Signing secret** and click **Save and start receiving**.

Step 11. **Expected:** the status reads **Waiting for the first webhook…** and the row shows **secret saved in Needl**.

4.C Test

Browser tab: Linear

Create an issue in any public team titled `Needl webhook test`. Then move it to **In Progress** (or any other state) and back.

Browser tab: Needl

Success: **Receiving events · last: Issue.create**, then **… last: Issue.update**. You can delete or cancel the test issue afterwards.

4.D Fixes

- **No New webhook / API settings visible:** you're not a workspace admin. Send a workspace admin 4.B steps 3–9 and paste the secret yourself.
- **Signature didn't match:** you copied a different webhook's secret. Open the Needl webhook in Linear, copy its **Signing secret** again, then in Needl click **Setup** → paste → **Replace secret**.
- **Old timestamp:** Linear allows 60 seconds. Make a new issue change instead of waiting for Linear's retries.
- **Still Waiting…:** the test issue's team is private and you chose all public teams. Use a public team.

5 LaunchDarkly

5.A Connect the official LaunchDarkly MCP to Claude Code

Endpoint `https://mcp.launchdarkly.com/mcp/launchdarkly` · HTTP · OAuth. Hosted MCP is **not** available on the EU or federal instances.

Terminal (Mac)

Step 1. Add:

```
claude mcp add --transport http --scope user \
  launchdarkly https://mcp.launchdarkly.com/mcp/launchdarkly
```

Step 2. Sign in:

```
claude mcp login launchdarkly
```

Browser: LaunchDarkly authorization page

Step 3. Check the account shown, then authorize.

Terminal (Mac)

Step 4. Run `claude mcp list`. **Expected:** `launchdarkly` … ✔ `Connected`.

Step 5. Run `claude`, then:

> Using only the launchdarkly MCP server, list my LaunchDarkly projects (name and key). Read only: do not create, change, toggle or delete anything.

Expected: your project names and keys. Type `/exit`.

The LaunchDarkly MCP's observability alert tools (`create-alert` and similar) are **not** Needl's webhook. Needl reads signed audit-log webhooks from **Integrations → Webhooks** only.

5.B Connect the LaunchDarkly webhook to Needl

Browser tab: Needl (Connections)

Step 1. On the **LaunchDarkly** card click **Connect** (or **Finish setup**).

Step 2. Click **Copy** next to **Webhook URL**.

Browser tab: LaunchDarkly

Step 3. Click the gear icon (Organization settings) in the left sidebar, then **Integrations**.

Step 4. Search for `Webhooks`, open it, and click **Add integration**. The **Create a webhook** panel opens.

Step 5. **Name**: `Needl`. **URL**: paste.

Step 6. Tick **Sign this webhook**. A **Secret** value appears; copy it.

Step 7. Leave the policy editor empty. By default LaunchDarkly then sends all flag changes in the **production** environment.

Step 8. **Don't save yet**: first do Steps 9–10 in Needl.

Copy & paste

LaunchDarkly **Create a webhook** → **Secret** → Needl dialog field **Signing secret**

Browser tab: Needl (dialog)

Step 9. Paste into **Signing secret** and click **Save and start receiving**.

Step 10. **Expected: Waiting for the first webhook…**.

Browser tab: LaunchDarkly

Step 11. Tick the Integration Terms and Conditions box, then click **Save settings**.

5.C Test

Browser tab: LaunchDarkly

LaunchDarkly has no test button. Create a boolean flag with key `needl-webhook-test` (no code reads it, so it's harmless). Switch to the **Production** environment and toggle the flag **On**, then **Off**.

Browser tab: Needl

Success: Receiving events · last: flag.updateOn (or another `flag.…` action). You can archive the test flag afterwards.

5.D Fixes

- **No Add integration button:** your role can't manage integrations. Ask a LaunchDarkly admin to do 5.B steps 3–11 with you.
- **Signature didn't match:** the secret in LaunchDarkly changed. Edit the webhook, copy **Secret**, then in Needl click **Setup** → paste → **Replace secret**.
- **Nothing arrives:** you toggled a flag in a non-production environment. Toggle it in Production, or add a policy that includes that environment.
- LaunchDarkly retries a failed delivery only once, so after a fix send a fresh toggle.

6 Sentry

6.A Connect the official Sentry MCP to Claude Code

Endpoint `https://mcp.sentry.dev/mcp` · HTTP · OAuth.

Terminal (Mac)

Step 1. Add:

```
claude mcp add --transport http --scope user \
  sentry https://mcp.sentry.dev/mcp
```

Step 2. Sign in:

```
claude mcp login sentry
```

Browser: Sentry authorization page

Step 3. Sign in to Sentry, check the organisation shown, then approve.

Terminal (Mac)

Step 4. Run `claude mcp list`. **Expected:** `sentry` … ✔ `Connected`.

Step 5. Run `claude`, then:

> Using only the sentry MCP server, list the Sentry organizations and projects I can access. Read only: do not resolve, assign, update or delete anything.

Expected: your organisation slug and project slugs. Type `/exit`.

Sentry's MCP doesn't create Custom Integrations, so 6.B is done in Sentry's own settings.

6.B Connect Sentry webhooks to Needl

Browser tab: Needl (Connections)

Step 1. On the **Sentry** card click **Connect** (or **Finish setup**).

Step 2. Click **Copy** next to **Webhook URL**.

Browser tab: Sentry

Step 3. Click the gear icon (**Settings**), then **Developer Settings** → **Custom Integrations**.

Step 4. Click **Create New Integration**, choose **Internal Integration**, and click **Next**.

Step 5. **Name**: `Needl`. **Webhook URL**: paste.

Step 6. Turn on **Alert Rule Action**. This lets your alert rules send to Needl later; it changes nothing by itself.

Step 7. **Permissions**: set **Issue & Event** to **Read**. Leave everything else at **No Access**.

Step 8. **Webhooks**: tick **issue**. Tick **error** as well **only** if your Sentry plan is Business or Enterprise (other plans don't offer error webhooks). Leave **comment** unticked.

Step 9. Click **Save Changes**. Creating an internal integration installs it on your organisation automatically.

Step 10. On the integration page, under **Credentials**, copy **Client Secret**. Don't copy the Client ID or a token.

Copy & paste

Sentry integration **Credentials** → **Client Secret** → Needl dialog field **Client Secret**

Browser tab: Needl (dialog)

Step 11. Paste into **Client Secret** and click **Save and start receiving**.

Step 12. **Expected: Waiting for the first webhook…**. If it briefly said **Webhooks are arriving…** before you saved, that was Sentry's installation notice, and it's fine.

6.C Test

Browser tab: Sentry

Open **Issues**, pick any existing issue in a non-critical project, click **Resolve**, then unresolve it.

Browser tab: Needl

Success: Receiving events · last: issue.resolved, then **… issue.unresolved**.

6.D Fixes

- **Create New Integration missing or disabled:** you lack the rights. Ask a Sentry organisation Owner or Manager to do 6.B steps 3–10 and paste the Client Secret into Needl while sharing their screen.
- **issue checkbox greyed out:** set **Issue & Event** to **Read** first.
- **Signature didn't match:** you pasted the Client ID or an auth token. Copy **Client Secret**, then in Needl click **Setup** → paste → **Replace secret**.
- **…isn't one Needl reads (installation.created):** that's the installation notice. Run the 6.C test.
- **No issues to resolve:** send one test error from a test app, or wait for a real issue.

7 PostHog

7.A Connect the official PostHog MCP to Claude Code

Endpoint `https://mcp.posthog.com/mcp` · HTTP · OAuth. Sign-in picks your US or EU region automatically.

Terminal (Mac)

Step 1. Add:

```
claude mcp add --transport http --scope user \
  posthog https://mcp.posthog.com/mcp
```

Step 2. Sign in:

```
claude mcp login posthog
```

Browser: PostHog authorization page

Step 3. Sign in to PostHog, check the organisation and project shown, then authorize.

Terminal (Mac)

Step 4. Run `claude mcp list`. **Expected:** `posthog ... ✔ Connected`.

Step 5. Run `claude`, then:

> Using only the posthog MCP server, tell me which PostHog organization and projects I can access (names and ids). Read only: do not create, change or delete anything.

Expected: your organisation and projects. Type `/exit`.

Webhook *sources* (data coming into PostHog) are not what Needl uses. Needl needs an outgoing **HTTP Webhook destination**, set up in 7.B.

7.B Connect a PostHog Webhook destination to Needl

You make the signing secret yourself, then paste it into both Needl and PostHog.

Terminal (Mac)

Step 1. Generate a Standard Webhooks secret:

```
echo "whsec_$(openssl rand -base64 32)"
```

Expected: one line starting with `whsec_`, about 50 characters long. Select the whole line and copy it (Cmd+C). Don't save it anywhere else.

Browser tab: Needl (Connections)

Step 2. On the **PostHog** card click **Connect** (or **Finish setup**).

Step 3. Under **How webhooks are verified**, keep **Signing secret** selected.

Step 4. Paste the `whsec_…` line into **Signing secret or header token** and click **Save and start receiving**. Keep the dialog open.

Step 5. Click **Copy** next to **Webhook URL**.

Browser tab: PostHog

Step 6. Open **Data pipelines** in the sidebar → **Destinations**. Click **+ New** → **Destination**, search `Webhook`, and click **+ Create** on **HTTP Webhook**.

Step 7. **Webhook URL**: paste. **Method**: **POST**.

Browser tab: Needl (dialog)

Step 8. Click **Copy** next to **Webhook body**. The body is shown below for reference; use the Copy button, not this PDF:

```
{
  "event": "{event}",
  "project_id": "{project.id}",
  "relationships": []
}
```

Browser tab: PostHog

Step 9. **JSON Body**: select everything in it, delete it, then paste.

Step 10. **Headers**: leave as is (`Content-Type: application/json`).

Terminal (Mac)

Step 11. Your clipboard now holds the body, not the secret. Select the `whsec_…` line from step 1 again and press **Cmd+C**.

Browser tab: PostHog

Step 12. **Signing secret**: paste.

Step 13. **Filters**: set the match to the single event `$pageview`. This keeps volume low; later, add the event names your Needl rules use.

Step 14. Click **Create & Enable**.

Copy & paste

Terminal `whsec_…` line → Needl **Signing secret or header token and** PostHog **Signing secret**
Needl **Webhook URL** → PostHog **Webhook URL**
Needl **Webhook body** → PostHog **JSON Body**

7.C Test

Browser tab: PostHog (destination page)

In **Testing**, click **Start testing** → **Test function**.

Browser tab: Needl

Success: Receiving events · last: *event name* (for example `$pageview`). Close the Terminal window afterwards so the secret isn't left on screen.

7.D Fixes

- **Your PostHog form has no Signing secret field** (PostHog Cloud may lag their source code). Use the fallback: in Needl click **Setup** → select **Authorization header** → click **Generate** → copy the value shown under **Paste this into the provider too** → **Save and start receiving** (or **Replace secret**). In PostHog, add a header under **Secret headers** (or under **Headers** if there is no Secret headers section) with key `Authorization` and value `Bearer ` plus the token: one space after `Bearer`, nothing else. This proves the sender but doesn't sign the body.
- **Destination needs an upgrade:** the HTTP Webhook destination is a paid template. Ask your PostHog billing admin to enable Data pipelines, or skip PostHog.
- **Wasn't in the expected format:** the **JSON Body** isn't exactly Needl's body. Paste it again from Needl's **Copy**.
- **Signature didn't match:** the two `whsec_` values differ. Generate a new one (step 1), paste it into Needl (**Replace secret**) and into PostHog **Signing secret**, then save the destination.
- **PostHog error "not valid base64" or "at least 24 bytes":** part of the secret is missing. Copy the full line again.

8 Intercom

8.A Connect the official Intercom MCP to Claude Code

Endpoint `https://mcp.intercom.com/mcp` (US workspaces) or `https://mcp.eu.intercom.com/mcp` (EU workspaces) · HTTP · OAuth. Australian workspaces are not supported. If your Intercom address starts with `app.eu.intercom.com`, use the EU command.

Terminal (Mac)

Step 1. Add **one** of these. US:

```
claude mcp add --transport http --scope user \
  intercom https://mcp.intercom.com/mcp
```

EU:

```
claude mcp add --transport http --scope user \
  intercom https://mcp.eu.intercom.com/mcp
```

Step 2. Sign in:

```
claude mcp login intercom
```

Browser: Intercom authorization page

Step 3. Check the workspace shown, then authorize.

Terminal (Mac)

Step 4. Run `claude mcp list`. **Expected:** `intercom` … ✔ `Connected`.

Step 5. Run `claude`, then:

> Using only the intercom MCP server, list up to 3 help center articles (titles only). Read only: do not create, update, reply or add notes.

Expected: article titles, or a reply saying there are no articles. Either one proves the connection. Type `/exit`.

Intercom's MCP has no webhook tools. Webhook topics are set up in the Developer Hub (8.B).

8.B Connect Intercom webhooks to Needl

Known risk. When you save, Intercom checks the URL with a `HEAD` request. Needl's current code answers `POST` on webhook URLs and returns 404 for `HEAD`. If Intercom refuses to save the URL (step 10), **stop**. Send the developer this: *"Needl /api/hooks/<id> must answer HEAD with 200 for Intercom's URL validation."* Don't try other URLs.

Browser tab: Intercom Developer Hub

Step 1. Open `https://app.intercom.com/a/apps/_/developer-hub`. Under **Your apps**, open your app. If you have none, click **New app**, name it `Needl`,

choose your workspace, and click **Create app**; this pre-installs it in that workspace.

Step 2. Open **Basic information** and copy **Client secret**.

Copy & paste

Intercom **Basic information** → **Client secret** → Needl dialog field **Client secret**

Browser tab: Needl (Connections)

Step 3. On the **Intercom** card click **Connect** (or **Finish setup**).

Step 4. Paste into **Client secret** and click **Save and start receiving**. (Do this first, because Intercom sends a test request as soon as you save on its side.)

Step 5. Click **Copy** next to **Webhook URL**.

Browser tab: Intercom Developer Hub (your app)

Step 6. In the left menu, under **Configure**, click **Webhooks**.

Step 7. Paste into the endpoint URL field.

Step 8. In the topics dropdown, add exactly these 7 topics:

- `conversation.user.created`
- `conversation.admin.replied`
- `conversation.admin.closed`
- `ticket.created`
- `ticket.updated`
- `contact.created`
- `contact.updated`

Step 9. If the page shows a missing-permission error, open **Configure → Authentication**, add the permission it names, save, and return to **Webhooks**.

Step 10. Click **Save**.

8.C Test

Browser tab: Needl

Right after you save, Intercom sends a ping. If the signature is correct, Needl shows **Connected and verified, but the last event isn't one Needl reads (ping).** That is the expected result. To send it again, use **Send a test request** on Intercom's Webhooks page if it is shown.

Browser tab: Intercom (workspace)

Create a test contact: open **Contacts** and add a new user or lead. Use the email `needl-test@example.com`.

Browser tab: Needl

Success: Receiving events · last: contact.created. You can delete the test contact afterwards.

8.D Fixes

- **Intercom won't save the URL:** see the red box above. This is a Needl code limitation, not your mistake.
- **Signature didn't match:** you copied the Client ID or access token, or a different app's secret. Copy **Client secret** from *this* app's **Basic information**, then in Needl click **Setup** → paste → **Replace secret**.
- **Subscription suspended** (private apps, after 7 days of errors): fix the cause, then click **Set live** on the Webhooks page.
- **MCP fails for an Australian workspace:** not supported by Intercom. Skip 8.A; 8.B still works.

9 Stripe (sandbox only)

9.A Connect the official Stripe MCP to Claude Code

Endpoint `https://mcp.stripe.com` · HTTP · OAuth. On Stripe's consent page you choose which accounts or sandboxes the client may use.

Terminal (Mac)

Step 1. Add:

```
claude mcp add --transport http --scope user \
  stripe https://mcp.stripe.com/
```

Step 2. Sign in:

```
claude mcp login stripe
```

Browser: Stripe OAuth consent page

Step 3. Select **only your sandbox**. Leave every live-mode account unselected. Then grant access.

Terminal (Mac)

Step 4. Run `claude mcp list`. **Expected:** `stripe` … ✔ `Connected`.

Step 5. Run `claude`, then:

> Using only the stripe MCP server, call get_stripe_account_info and tell me the account name and whether it is a sandbox. Read only: do not create, update, refund or delete anything.

Expected: the account name and confirmation that it is a sandbox. Type `/exit`.

Stripe's MCP *could* create webhook endpoints through `stripe_api_write`. This manual sets them up in Workbench on purpose, because the Dashboard shows the signing secret directly. Never use the MCP for refunds or payouts; Stripe asks for human approval on such writes anyway.

9.B Connect a Stripe event destination to Needl

Browser tab: Needl (Connections)

Step 1. On the **Stripe** card click **Connect** (or **Finish setup**).

Step 2. Click **Copy** next to **Webhook URL**.

Browser tab: Stripe Dashboard (sandbox)

Step 3. Check that the account picker (top left) shows your **sandbox**. Open `https://dashboard.stripe.com/webhooks` (Workbench → **Webhooks**).

Step 4. Click **Create an event destination**, then select **Your account**.

Step 5. Payload format: choose **Snapshot** (**not** Thin). Leave the API version as preselected.

Step 6. Select these 9 events (type each into the search box and tick it):

- `payment_intent.succeeded`
- `payment_intent.payment_failed`
- `charge.succeeded`
- `charge.failed`
- `charge.dispute.created`
- `invoice.paid`
- `invoice.payment_failed`
- `customer.subscription.updated`
- `customer.subscription.deleted`

Step 7. Click **Continue**, choose **Webhook endpoint**, then click **Continue**.

Step 8. **Endpoint URL**: paste. Description: `Needl`. Click the final create button.

Step 9. On the destination's settings page, click **Reveal secret** and copy the value that starts with `whsec_`.

Copy & paste

Stripe destination page → **Reveal secret** (`whsec_…`) → Needl dialog field **Signing secret**
Not the API keys `sk_…` / `pk_…` / `rk_….`

Browser tab: Needl (dialog)

Step 10. Paste into **Signing secret** and click **Save and start receiving**.

Step 11. **Expected: Waiting for the first webhook…**.

9.C Test

Browser tab: Stripe Dashboard (sandbox) → Workbench → Shell tab

Type this and press **Return**. Shell is read-only in live mode, so this only works in a sandbox:

```
stripe trigger payment_intent.succeeded
```

Expected in Shell: `Trigger succeeded!`

Browser tab: Needl

Success: the status reads **Receiving events · last:** followed by one of these: `payment_intent.succeeded` or `charge.succeeded` (the trigger produces both).

Optional, after setup: run `claude` and paste this to cross-check from the MCP side:

> Using only the stripe MCP server in my sandbox, list my event destinations / webhook endpoints and show the URL and enabled events of the one whose URL starts with https://needl.dev. Read only: change nothing.

9.D Fixes

- **Needl says `Stripe webhook signing secrets start with whsec_ (not the sk_ or pk_ API keys).`** You pasted an API key. Use **Reveal secret** on the destination page.
- **Wasn't in the expected format:** you created a **Thin** destination. Create a new **Snapshot** destination with the same URL, put its secret into Needl (**Setup** → **Replace secret**), then delete the Thin destination in Stripe.
- **Signature didn't match:** you used the secret printed by `stripe listen` (local CLI), which is a different secret. Use the destination's **Reveal secret**.
- **`stripe trigger` refused in Shell:** you're in live mode. Switch to the sandbox.

10 After setup

10.1 Final checklist

Tool	A: `claude mcp list` shows	B/C: Needl row shows
GitLab	`gitlab … ✔ Connected`	existing green row (unchanged)
Linear	`linear … ✔ Connected`	**Receiving events · last: Issue….**
LaunchDarkly	`launchdarkly … ✔ Connected`	**Receiving events · last: flag….**
Sentry	`sentry … ✔ Connected`	**Receiving events · last: issue….**
PostHog	`posthog … ✔ Connected`	**Receiving events · last:** *event*
Intercom	`intercom … ✔ Connected`	**Receiving events · last: contact.created**
Stripe	`stripe … ✔ Connected` (sandbox)	**Receiving events · last: payment_intent….**

10.2 General MCP fixes

- `! Needs authentication` → run `claude mcp login <name>`.
- A notice in Claude Code about an expired sign-in → type `/mcp`, select the server, choose **Re-authenticate**.
- ✘ `Failed to connect` → check the URL with `claude mcp get <name>`. If it's wrong, run `claude mcp remove <name> --scope user` and add it again from the chapter.
- The browser doesn't open → copy the URL Claude Code prints into your browser.
- `already exists` when adding → the name is taken. Run `claude mcp get <name>`; if it points to the URL in this manual, skip adding and just sign in.
- Removing a server's access completely: `claude mcp remove <name> --scope user`. You can also revoke the app in the tool's own account settings (Stripe: user settings → **OAuth sessions** → **Revoke access**).

10.3 Safe use of MCP later

When a tool calls for it, Claude Code asks before using it. Approve calls one at a time, and don't choose "always allow" for tools that write. A safe pattern for any later request is to end it with: *"Read only unless I confirm a specific change."*

10.4 What this manual could not verify

- It was written without signing in to any of the seven tools, so no live screen was clicked through. Labels come from the Needl source code (commit 16eec52) and official vendor docs checked on 2026-10-19. A few labels (Sentry form, LaunchDarkly **Secret** field, Intercom **Send a test request**) come from well-known third-party guides, because the vendor docs don't name them. Vendors rename buttons; if one differs slightly, choose the closest match with the same meaning.
- Exact text printed by `claude mcp add` and `claude mcp login` may vary by Claude Code version. The checked version was 2.1.302.
- Intercom's HEAD validation against Needl (red box in Chapter 8) was found by read-

ing Needl's code and was not tried live.

- The PostHog **Signing secret** field exists in PostHog's open-source template. Whether your PostHog Cloud project shows it, and which plan it needs, was not checked live; 7.D gives the fallback.
- Every success check proves that *delivery* works. It doesn't prove that a rule or a GitLab flow ran.

10.5 Official sources

- Claude Code MCP: https://code.claude.com/docs/en/mcp
- GitLab MCP: https://docs.gitlab.com/user/model_context_protocol/mcp_server/
- GitLab webhooks: https://docs.gitlab.com/user/project/integrations/webhooks/
- Linear MCP: https://linear.app/docs/mcp
- Linear webhooks: https://linear.app/developers/webhooks
- LaunchDarkly MCP: https://launchdarkly.com/docs/home/getting-started/mcp-hosted
- LaunchDarkly webhooks: https://launchdarkly.com/docs/home/infrastructure/webhooks
- Sentry MCP: https://mcp.sentry.dev/
- Sentry integration webhooks: https://docs.sentry.io/organization/integrations/integration-platform/webhooks/
- PostHog MCP: https://posthog.com/docs/model-context-protocol/claude-code
- PostHog webhook destination: https://posthog.com/docs/cdp/destinations/webhook
- Intercom MCP: https://developers.intercom.com/docs/guides/mcp
- Intercom webhooks: https://developers.intercom.com/docs/webhooks/setting-up-webhooks
- Stripe MCP: https://docs.stripe.com/mcp
- Stripe webhooks: https://docs.stripe.com/webhooks
- Stripe sandboxes: https://docs.stripe.com/sandboxes

Full evidence map with check dates: `docs/setup-manual/SOURCES.md` in the Needl repository.